

Computer-use capabilities for systems that were never meant to be automated.

A model learns a job inside a legacy bank application **once**.

What it learns becomes a typed, versioned, reviewable capability that replays deterministically —

and that an AI agent calls by name, for free, forever.

discovery: 1 model run · replay: ~4.7s, 0 model calls · 5/5 identical outcomes

~8,500 US institutions run software with no usable API.

CONCENTRATED

70%+

of banks sit on Fiserv, Jack Henry or FIS. 520 banks on SilverLake alone; ~715 credit unions on Symitar.

OLD

1968–1989

Origin dates of the ten first-generation cores still in service — COBOL, RPG, Progress. Over 50 are still supported in the US.

STUCK

61%

have been with their provider over ten years. 46% are dissatisfied. They will buy an overlay; they will not buy a replacement.

And the API is not the escape hatch people assume: the ABA told the OCC in 2025 that core providers restrict API access by **“charging high fees, requiring excessive implementation times, or failing to support API connections altogether.”**

Point a computer-use model at the screen every time?

NOT REPRODUCIBLE

~1 in 6

Frontier models now beat the human baseline on OSWorld (72.36%) and still fail roughly one task in six. That is a fine assistant and an unacceptable unattended teller.

NOT AFFORDABLE

per click

A model round-trip for every step, times thousands of exception items a day.

NOT AUDITABLE

SR 11-7

The *institution* must perform its own outcomes testing on a vendor's model.

It has been tried at scale and withdrawn. OpenAI's Operator launched Feb 2025 and shut down Aug 2025. Adept raised \$415M+ and was absorbed into Amazon. General-purpose agentic browsing has not found a durable enterprise buyer.

THE THESIS

**You cannot outcomes-test
a system that does not
produce the same output
twice.**

In this market determinism is not a nice-to-have. It is the precondition for the examiner signing off at all.

So put the model where it is genuinely good — reading an unfamiliar screen *once* and working out what a trained operator would do — and take it out of the path where determinism is the requirement.

the model discovers → the artifact is the capability → deterministic replay is how it runs in production

The model chooses **which** control. Swivel decides **how it is described**.

WHY NOT ASK THE MODEL FOR A SELECTOR

- Quality varies run to run — artifact durability becomes non-deterministic.
- It cannot see geometry, so it reaches for the DOM id: the *weakest* evidence available.
- It does not know which on-screen value came from a parameter, so it hardcodes one member's number into a capability meant to serve all of them.

WHAT IS ACTUALLY STABLE

What a human operator uses: what the control **is**, what it is **called**, what it is **near**.

accessible name	40
grid row matched by value	45
caption beside an unlabelled box	25
generated-id <i>pattern</i>	18
literal DOM id	12
ordinal position	5

Scored at replay time. Below threshold, or too close to call, **the engine refuses rather than guessing** — and reports which evidence went missing. That report is also the earliest available drift signal.

“No such member” is an answer, not a crash.

Conflating the two is the most common way this kind of system goes wrong.
The result type makes it impossible.

RESULT	MEANING	THE CALLER
success	flow completed	reads typed outputs
business_outcome	RECORD_NOT_FOUND · NOT_AUTHORIZED · VALIDATION_FAILED · SYSTEM_UNAVAILABLE_EOD · RECORD_LOCKED	branches on the code; retries only if marked retryable
escalated	a human drove the live session; here is what they did	verifies before acting
failed	an incident — which step, expected, observed, evidence	pages someone

Signals are authored once per vendor product, not per capability.

Symitar times out the same way for every credit union running it. Ten capabilities share one definition of “your session expired”; when the vendor changes that screen, one profile changes and every capability inherits the fix.

The session stays alive. Control moves.

NOT A TICKET QUEUE

When the engine refuses to guess, it **pauses** — and the browser session is still there, mid-flow, with whatever the application has built up server-side. The operator gets a live screencast of *that* session and drives it. A handoff that starts fresh hands the operator the problem of getting back to where the automation was.

WHO IS IN CONTROL IS ENFORCED

A single-writer lease, wrapped around the surface, so acting without the token is impossible — not merely discouraged. Observation stays open to everyone; only mutation is gated. A stale tab or a second operator cannot interleave keystrokes into a funds-transfer form.

Measured, not described: the same artifact replayed at a second institution refused one control it could not identify, escalated, a human did that step by hand in the live session, handed control back — and the automation finished the remaining steps itself. Every forwarded click is on the run's evidence chain.

Next: promote those recorded rescues into reviewed step patches, so each escalation is the last of its kind across every tenant.

One artifact. Hundreds of institutions.

Many tenants run the *same* vendor product, branded and versioned differently. Rebuilding per tenant is $100 \times 20 \times 10 = \mathbf{20,000 \text{ recordings}}$. As base capabilities plus overlays it is **~200 recordings** and 20,000 small, reviewable patches.

	PINE RIDGE FCU	HARBOR POINT BANK
Product version	Meridian 9.2.14	Meridian 10.1.3
Vocabulary	“Member #”	“Customer ID”
Generated control ids	ctl00_Main_*	ctl00_cphMain_*
Results grid	Member #, Name, SSN...	Segment inserted before Name
Balances grid	shown	collapsed behind a control
Overlay needed	—	one step patch

Everything else generalised on its own: the caption-based target resolved against “Customer ID”, and the results row matched by member number *despite the inserted column* — which is exactly what breaks a column-index locator.

Sell the capability, not the seat.

	TRADITIONAL RPA	LLM IN THE LOOP	SWIVEL
Build one automation	weeks of dev time	—	one discovery run
Cost per execution	licence-amortised	a model call per step	\$0 — no model in the loop
Time per execution	seconds	minutes	~4.7s, measured
Same output twice	yes	no	yes — 5/5 measured
Vendor ships a new version	rebuild	re-reason every run	re-record once; all tenants inherit
Institution #2 on the same product	rebuild	—	one overlay

UiPath publishes almost no pricing — but it does publish **Self-Healing at \$5,000 per 5,000 heals**. The category leader sells selector repair by the unit. Only **3–4%** of organisations ever scaled past 50 robots.

Sources: UiPath published pricing · Deloitte Global RPA Survey. Widely-quoted RPA failure and maintenance percentages are excluded here: they do not trace to a primary source.

Every number below came out of the repository.

MEASURED

- **2 capabilities** discovered by a real model against a live legacy UI
- **~4.7s replay, 0 model calls**, all targets at score 100
- **5/5 identical outcomes**, 0% flakiness
- Business outcomes, recoveries and safety refusals all exercised end to end
- **Live CDP takeover**: operator drove the paused session; both hand-back outcomes on the record
- **The control experiment**: the same artifact at the second institution with the overlay *off* — it refuses at score 0 rather than reading the nearest cell
- **173 tests**, almost all without a browser — which is the evidence the perception seam is real

WHAT I WOULD BUILD NEXT

- **Promote human rescues into capabilities** — the recording already exists
- **A UIA desktop surface**, to prove the seam against a thick client rather than argue for it
- **Fleet drift monitoring** — alert on falling targeting scores during a vendor rollout, before an outage
- **Bounded single-step LLM repair**, policy-gated, with escalation as the fallback